

SYMPTOM & MEDICATION JOURNAL - PROJECT REPORT

Submitted by: - Somya Singh

Registration no.: - 26BCE11150

Course: - Introduction to Problem Solving And Programming

Platform: - VITyarthi - Build Your Own Project

1. Introduction: -

The Symptom & Medication Journal is a digital health tracking utility created to replace manual, paper-based medical logs. This version of the application uses Python's Tkinter library to provide a graphical interface, replacing the original command-line menu with clickable forms, dropdowns, and a live, always-visible journal list. The application lets a user record medication intake and symptom occurrences with exact timestamps, offering a simple, structured way to track personal health history that can better inform conversations with a healthcare provider.

2. Problem Statement: -

Management of chronic conditions, such as migraines, arthritis, or allergies, depends heavily on human memory, which is subjective and prone to errors. Patients often cannot answer questions that are specific and clinically relevant to a doctor, such as “How many times did you take this painkiller last month?” or “How much time elapsed between the start of a symptom and the medicine taking effect?” Without accurate, easy-to-review data, it becomes difficult to measure how well a treatment is actually working.

3. Functional Requirements: -

Core capabilities provided by the system include:

- **Item Management:** Users can add a new medication or symptom to their tracking list, choosing its category (Medication/Symptom) from a dropdown menu.
- **Event Logging:** Users can select an item from the journal list and log an occurrence for it; the system automatically captures the current timestamp.
- **Live Summary View:** The same journal list always shows the total occurrence count and the most recent timestamp for every tracked item, without a separate menu step.

- **Item Removal:** Users can delete an item they no longer want to track directly from the list.
- **Input Validation:** The system normalizes names with `.strip().title()` so “Advil” and “advil” are treated as the same entry, and blocks empty names or missing selections with clear warning pop-ups.

4. Non-Functional Requirements: -

- **Usability:** The GUI uses labeled fields, a dropdown for category selection, and click-based list selection, removing the need to remember menu numbers or item indexes.
- **Reliability:** The system avoids crashes on invalid input - such as trying to log an event with nothing selected - by checking selections before acting and showing a warning instead.
- **Data Integrity:** Timestamps are generated automatically by the system and cannot be typed in or falsified by the user.
- **Maintainability:** The interface code (`create_widgets`) is kept separate from the logic (`add_item`, `log_occurrence`, `delete_item`, `refresh_list`), so each piece can be updated independently.

5. System Architecture: -

The application follows a simple layered architecture:

- **Presentation Layer (GUI):** Built with Tkinter - an entry field, a category dropdown, a journal Listbox, and buttons for Add / Log / Delete.
- **Logic Layer:** Python functions that add new items, increment counts, fetch timestamps, and remove items from the tracked list.
- **Data Layer:** A list of dictionaries held in memory for the duration of the session, with one dictionary per tracked item (name, category, count, last_logged).

6. Design Decisions & Rationale: -

- **Language Choice (Python):** Chosen for its readability and its built-in Tkinter library, which needs no extra installation for a desktop GUI.
- **GUI over CLI:** The original command-line menu was replaced with a GUI to make the application accessible to non-technical users who are not comfortable typing menu numbers.

- **Listbox for the Journal:** A single Listbox was used to double as both the item list and the summary view, keeping the interface uncluttered and avoiding a separate “View Summary” screen.
- **In-Memory List:** A list of dictionaries was kept as the data store (rather than a database) to keep the logic close to the original console version and easy to follow.

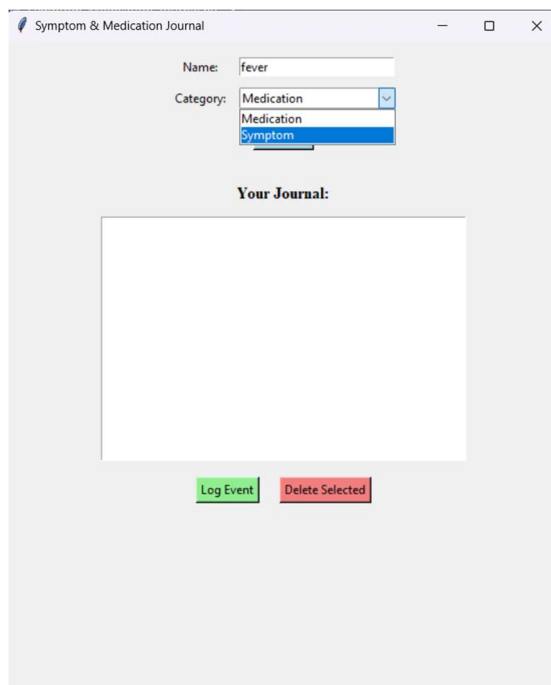
7. Implementation Details: -

The core logic relies on Python's datetime module for temporal tracking.

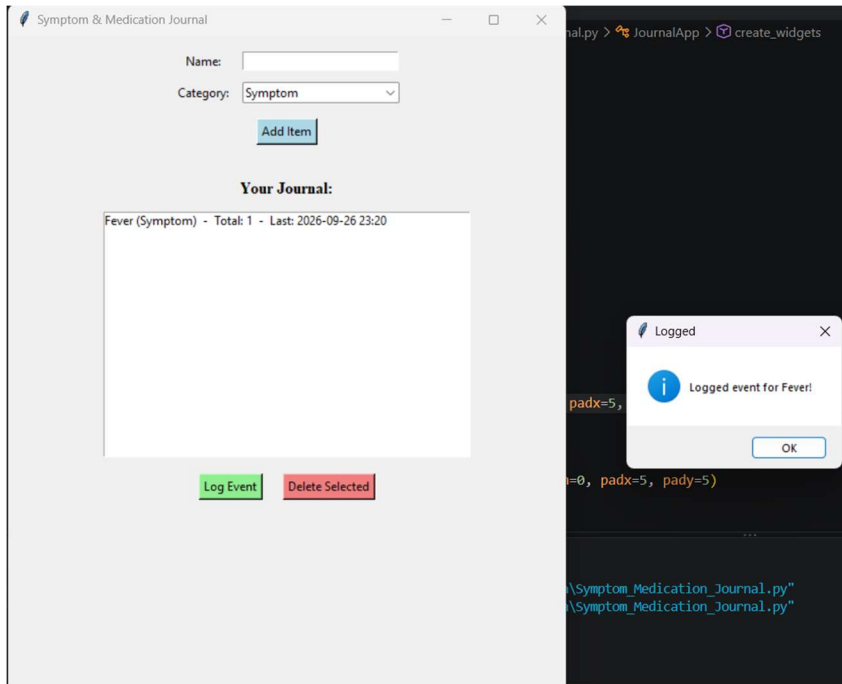
- **Input Sanitization:** The `.strip().title()` method chain is applied to every name entered, so extra spaces and inconsistent capitalization do not create duplicate-looking entries.
- **Selection-Based Actions:** Logging and deleting both work by reading the selected row's position in the Listbox and using it to find the matching dictionary in the data list.
- **exportselection=False:** Set on the journal Listbox so its selection behaves predictably and is not accidentally cleared by interactions elsewhere in the window.

8. Screenshots / Results: -

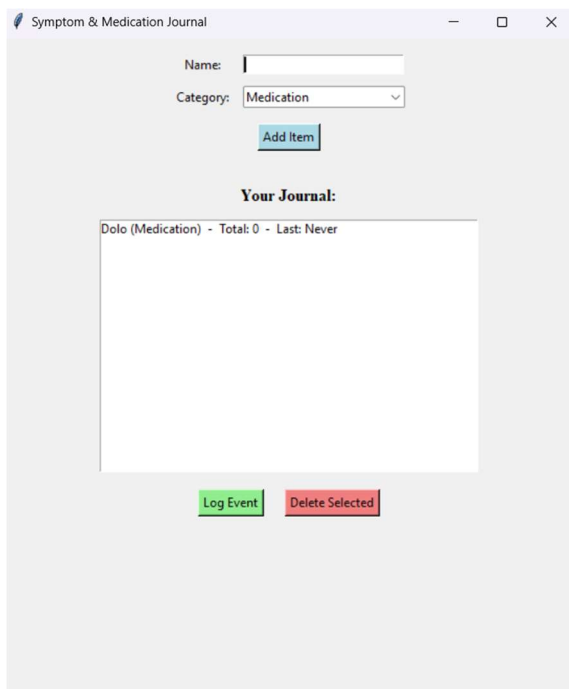
1. Adding a new item with its name and category



2. Logging an event for a selected item



3. Viewing the live journal with counts and last-logged times



9. Testing Approach

- **Unit Testing:** Simulated adding, logging, and deleting items to confirm each action updates only the correct entry in the data list.

- **Boundary Testing:** Verified that clicking Log Event or Delete with nothing selected shows a warning instead of crashing.
- **Format Testing:** Verified that timestamps are recorded and displayed accurately to the minute after logging an event.

10. Challenges Faced: -

- Rebuilding the console menu's three actions (Add, Log, View Summary) as GUI equivalents without losing any of the original functionality.
- Matching a clicked row in the Listbox back to the correct dictionary in the data list, since the Listbox only stores displayed text, not the underlying record.
- Deciding how to represent “View Summary” in a GUI - solved by keeping the journal list always up to date instead of requiring a separate button.

11. Learnings & Key Takeaways: -

- Learned how to translate a console menu's logic into GUI event handlers driven by button clicks.
- Understood how Tkinter's Listbox selection can be matched back to a parallel Python data structure by index.
- Reinforced the importance of automatic timestamps for producing reliable, trustworthy health data.

12. Future Enhancements: -

- **Data Persistence:** Save the journal to a file (CSV or SQLite) so entries are not lost when the program closes.
- **Visualizations:** Integrate Matplotlib to show symptom frequency and medication timing trends over time.
- **AI Analysis:** Apply correlation or regression techniques (e.g. with Pandas and Scikit-learn) to suggest potential links between medications and symptom relief.

13. References: -

- Python 3 Documentation: docs.python.org
- Tkinter Reference: <https://tkdocs.com/>
- "Automate the Boring Stuff with Python" by Al Sweigart
- VITyarthi Project Guidelines